

Research decision records (D01-D06, PC01)

source: research/decisions/ — narrative decision records, v1.4.0

Decision D01 — Research domain scope

- Date: 2023-09-30
- Status: applied (superseded by none)
- Contributor: Applicant

Background

At project inception the method could target any high-volume billing environment. The choice of domain determines the event dictionary, the data contract and the realism targets of the data preparation.

Options considered

1. General healthcare billing (hospitals, clinics, mixed service lines).
2. Clinical laboratory billing.
3. Commercial B2B invoicing (no payer adjudication).

Decision

Clinical laboratory billing (option 2).

Rationale

Laboratory billing combines high claim volume with multi-payer adjudication, frequent corrections and versioned bills, which exercises the reconstruction problem; it also matches the applicant's professional context in billing-department business analysis. Deep Claim (arXiv:2007.06229) and the invoice-lateness literature (Zeng et al. 2007, Schoonbee 2022) were reviewed as anchors before this choice (research/prior_work.csv: PW06-PW08).

Corresponding implementation

- Service mix and pricing in src/lab_billing/schema/data_preparation.py (SERVICE_MIX: panels, chemistry, molecular, anatomic pathology, toxicology, microbiology).
- Site contracts in data/reference/org_profiles.csv.

Verification

Domain realism checked in the data-quality review of the first prepared dataset

run (amount distributions, payer mix, weekday rhythms).

Decision D02 — Linkage philosophy

- Date: 2023-10-02
- Status: applied
- Contributor: Applicant

Background

Multi-source billing records contain genuinely ambiguous cases (same native number, no explicit predecessor, close amounts and dates). A linker can either force a single best guess or preserve uncertainty.

Options considered

1. Forced single-best link for every candidate pair.
2. Uncertainty-preserving tiers: accepted / rejected / needs-review.

Decision

Uncertainty-preserving tiers (option 2).

Rationale

Forced picks inject errors silently; downstream amount math and payment prediction then compound them. The possible-link disposition in Fellegi & Sunter (1969) is the theoretical basis for an explicit review tier (research/prior_work.csv: PW01).

Corresponding implementation

- src/lab_billing/reconstruct/rules.py: R2 version chains and R3 weak-evidence candidates with uniqueness gating; multi-candidate pairs are routed to review_queue instead of being merged.
- Review queue semantics documented in the method report, section 5.

Verification

- Acceptance tests T04 (multi-candidate needs review) and T05 (broken

reference flagged) in tests/test_module_a.py.

- Hidden-truth evaluation: recall 0.99 / precision 0.76 with a 9.6%

refusal ratio (validation_report.json) — high recall with controlled

review volume was the goal of this choice.

Decision D03 — Participating site contracts for the study data contract

- Date: 2023-10-06
- Status: applied
- Contributor: Applicant

Background

A single-site dataset would let the tool silently learn site-specific

idiosyncrasies. Multi-site structure forces cross-org identifier

isolation and tests generalization assumptions.

Options considered

1. A single participating site.
2. Three sites with distinct ownership models.
3. More than three sites.

Decision

Three sites (option 2): an independent regional laboratory (TRI), a

laboratory within an international diagnostics group (CPL), and a

not-for-profit health-system laboratory (NDX).

Rationale

Three ownership models across three geographies make cross-org

duplicate-number handling and payer-mix variation structurally testable

without an unmanageable preparation surface. Site tokens are used

throughout the data contract; the profiles live in

data/reference/org_profiles.csv.

Corresponding implementation

- configs/reference.json org_tokens = ["CPL", "TRI", "NDX"].
- Per-site timezones with DST in data_preparation.py (ORG_TZ).

Verification

- Acceptance test T02 (cross-org same number never links),
tests/test_module_a.py.
- T06-T08 amount/allocations scenarios run across sites.

Decision D04 — Prediction target

- Date: 2023-10-09
- Status: applied (real business confirmation still pending; unknowns
U03/U04)
- Contributor: Applicant

Background

"Will we get paid" is not directly observable from billing systems:

final cash depends on banking events outside the data. An operationally actionable target must be labelable from the data itself.

Options considered

1. Final collection / full settlement prediction.
2. First observable positive payment allocation or no-payment closure within a horizon.
3. Total collected amount regression.

Decision

Option 2: first observable positive allocation or explicit no-payment closure within $H=30$ days after landmark t (first observable submission + 7 days), with a third still-open class.

Rationale

Deep Claim's response-date framing (PW06) and competing-risk theory (Fine & Gray 1999, PW04) support a first-event formulation. Final collection depends on unobservable events; the first observable event is both actionable and labelable. The three classes are mutually exclusive and exhaust the space, which makes the probability constraints testable (T16).

Corresponding implementation

- src/lab_billing/forecast/snapshot.py (SnapshotBuilder._label: first observable payment / close / still-open; censored when the observation window ends before H).
- configs/reference.json prediction block (landmark_days=7, horizon_days=30).

Verification

- T12 (censored rows never become negatives), T19 (first event only, reopen does not rewrite), tests/test_module_b.py.
- Outcome mix on the reference run: 539 payment / 488 still-open / 8 close / 2 censored snapshots.

Decision D05 — Modeling frame for module B

- Date: 2023-10-12
- Status: applied
- Contributor: Applicant

Background

Observation in billing exports is irregular (late arrivals, gaps, DST), so continuous-time survival modeling would need assumptions the data cannot support. An interval formulation matches the daily operational cadence.

Options considered

1. Continuous-time proportional-hazards / Fine-Gray regression.
2. Discrete-time competing-event model: per-interval conditional probabilities (payment / close / none) with product-rule cumulative curves.
3. Static binary classifier at t (payment vs not).

Decision

Discrete-time competing-event model (option 2).

Rationale

The product rule $S(0)=1$; $F_payment(h)=\sum S(k-1) p_payment(k)$; $S(h)=S(h-1) p_none(h)$ yields three probabilities that sum to one at every

step — a testable invariant (T16). Static classifiers (option 3) cannot express the time structure of when the first event arrives, which is the operational question.

Corresponding implementation

- `src/lab_billing/forecast/models.py`: B0/B1/C1, `cumulative_curves`, `sum-to-one` enforcement.
- protocol v1.0 section Module B.

Verification

- T16 (probabilities non-negative, sum to one, cumulative curves correct), `tests/test_module_b.py`.
- Locked-test snapshot metrics: B1 log-loss 0.669 vs B0 0.736.

Decision D06 — Evaluation split

- Date: 2023-10-20
- Status: applied
- Contributor: Applicant

Background

Random splits leak: version chains and later events of the same lineage can land in both train and test. Temporal grouping by lineage prevents this.

Options considered

1. Random train/test split at record level.
2. Temporal split grouped by lineage (60/20/20).
3. Group k-fold by org.

Decision

Temporal 60/20/20 grouped by lineage (option 2), test portion locked.

Rationale

The same lineage and its corrected versions must never span splits (T14). Temporal ordering reflects deployment reality (models only see the past). Org-level k-fold was deferred; the multi-site structure is

instead exercised by the multi-site data contract (D03).

Corresponding implementation

- `src/lab_billing/forecast/pipeline.py` `temporal_grouped_split`.
- Leakage guards T10/T13/T14/T15.

Verification

- T14 (no lineage across splits), T15 (preprocessing fitted on train only), `tests/test_module_b.py`.
- Split sizes on the reference run: 12947 train / 4478 test interval rows.

Protocol Change PC01 — Interval granularity 1 -> 5 days

- Date: 2024-08-06
- Status: applied (protocol v1.1)
- Contributor: Applicant

Background

The first module B run (2024-08-05) at 1-day granularity produced unusable models: payment events concentrate around 10-30 days after submission, so 1-day intervals have >95% none-rows and the conditional probabilities are numerically unstable (B1 snapshot log-loss 11.3 vs B0 0.74).

Options considered

1. Keep 1-day intervals and add heavy class reweighting.
2. Change interval granularity to 5 days (H=30 becomes 6 intervals).
3. Move to a continuous-time model.

Decision

Option 2, recorded as a protocol change (not a silent edit).

Rationale

5-day intervals raise per-interval event rates into a learnable range while preserving within-horizon discrimination. Reweighting alone (option 1) distorts absolute probabilities — this was later confirmed in the balanced-class-weighting defect fixed in 2024-12. The change is versioned in `research/protocol_changes.csv` so the frozen protocol and

the implementation stay in sync.

Corresponding implementation

- configs/reference.json prediction.interval_days = 5.
- src/lab_billing/forecast/snapshot.py and models.py interval handling.
- research/protocol_v1.json v1.1 with change_history.

Verification

- Post-change locked-test metrics: B1 snapshot log-loss 0.669 (B0 0.736); C1 shows no incremental value (reported as-is).

Research decision records (narrative index)

These records expand research/research_decisions.csv into a

question-and-answer form: what was decided, what the alternatives were,

why this choice, where it lives in the code, and how it was verified.

Each entry traces to specific files and tests so the decision -> code ->

test chain is auditable.

- D01 2023-09-30 domain scope -> data_preparation.py SERVICE_MIX
- D02 2023-10-02 linkage philosophy -> reconstruct/rules.py, tests T04/T05
- D03 2023-10-06 site contracts -> configs/reference.json, org_profiles.csv
- D04 2023-10-09 prediction target -> forecast/snapshot.py, tests T12/T19
- D05 2023-10-12 modeling frame -> forecast/models.py, test T16
- D06 2023-10-20 evaluation split -> forecast/pipeline.py, tests T14/T15
- PC01 2024-08-06 interval granularity -> protocol v1.1, protocol_changes.csv